Point Distribution

Q1: Clustering Metrics [4pts]

Q2: EM Algorithm [15pts + 1% Bonus For All]

1. Performing an EM Update [15pts]
2. Gradient Ascent Comparison [1% Bonus For All]

Q3: Numeric Stability [6pts]

Q4: Alternate Implementation [5pts]

Q5: Semi-supervised Learning [2% Bonus For All]

Q6: K-Selection Problem [3% Bonus For All]

Points Totals:

- Non-Programming Base Points: 30pts
- Non-Programming Bonus for All: 6%
- For programming related points, see the Jupyter notebook.

1 Clustering Metrics [4pts]

In the Programming assignment (section 1.4), you should have implemented two clustering evaluation metrics: ARI and Silhouette Coefficient. Keeping those implementations in mind, answer the following questions.

a. For each metric, note the range of possible values, then describe what would constitute a "good" score and a "bad" score. [2pts]
ARI

- Range: -1 to 1.
- Good Score: A score close to 1 indicates that the predicted clusters are almost identical to the ground truth clusters.
- Bad Score: A score around 0 indicates random clustering, and a negative score implies performance worse than random chance.

Silhouette Coefficient

- Range: -1 to 1.
- Good Score: A score close to 1 is good, meaning that a data point fits well within its assigned cluster.
- Bad Score: A score close to -1 indicates points are likely assigned to the wrong clusters. A score near 0 indicates overlapping clusters.

b. Identify the most notable difference between these two metrics and explain why this distinction is important when evaluating clustering results. [2pts]

The most notable difference is that the ARI is an external evaluation metric, whereas the Silhouette Coefficient is an internal evaluation metric. (relying solely on the raw data and predicted cluster). This is crucial as ground truth lables are not usually accessible in the real world settings. Therefore, internal metrics are very practical for evaluating the quality of the clustering on unlabeled data, and external metrics are useful for benchmarking the performance of the algorithm on the labled data.

2 EM Algorithm [15pts + 1% Bonus For All]

2.1 Performing an EM Update [15pts]

Consider a univariate Gaussian Mixture Model (GMM) with two components. We'll add a slight variation: each component's mean and variation is parametrized together. In a normal GMM these are optimized separately, but this will save you some algebraic trouble. Our model is defined by the following parameter set $\theta = \{\alpha, \nu, \omega\}$, with:

$$\boldsymbol{z} = \begin{cases} 0 & \text{with } p = \alpha \\ 1 & \text{with } p = 1 - \alpha \end{cases}$$

$$p(x \mid \boldsymbol{z} = 0, \theta) = \mathcal{N}(x; \mu = 2\nu, \sigma^2 = 5\omega) = \frac{\exp\left(-\frac{(x-2\nu)^2}{10\omega}\right)}{\sqrt{10\pi\omega}}$$

$$p(x \mid \boldsymbol{z} = 1, \theta) = \mathcal{N}(x; \mu = 6\nu, \sigma^2 = 4\omega) = \frac{\exp\left(-\frac{(x-6\nu)^2}{8\omega}\right)}{\sqrt{8\pi\omega}}$$

Let's use this GMM to model the underlying distribution of a dataset $d = \{d^{\{1\}}, d^{\{2\}}, \cdots, d^{\{N\}}\}$. We aren't sure that it's a good fit yet, so we'll do an EM step to refine the parameters' (α, ν, ω) values.

a. Express the marginal probability density of the model $p(x \mid \theta)$ for an arbitrary value x. You may leave your answer in terms of $\mathcal{N}(x; \dots)$, no need to expand out Gaussians yet. Show your work. [2pts]

Hint

The rule of total probability may help.

using lotp:

$$p(x \mid \theta) = p(z = 0 \mid \theta)p(x \mid z = 0, \theta) + p(z = 1 \mid \theta)p(x \mid z = 1, \theta)$$

Substituting the parameters:

$$p(x \mid \theta) = \alpha\mathcal{N}(x; 2\nu, 5\omega) + (1 - \alpha)\mathcal{N}(x; 6\nu, 4\omega)$$

b. **E-Step:** Compute the posterior probability that a given arbitrary value x belongs to either component, the responsibilities of that value, $p(z = 0 \mid x, \theta)$ and $p(z = 1 \mid x, \theta)$. You may leave your answer in terms of $\mathcal{N}(x; \dots)$, no need to expand out Gaussians yet. Show your work. [4pts]

Hint

Bayes's rule may help.

Using Bayes' rule:

$$p(z = k \mid x, \theta) = \frac{p(x \mid z = k, \theta)p(z = k \mid \theta)}{p(x \mid \theta)}$$

For $z = 0$:

$$p(z = 0 \mid x, \theta) = \frac{\alpha\mathcal{N}(x; 2\nu, 5\omega)}{\alpha\mathcal{N}(x; 2\nu, 5\omega) + (1 - \alpha)\mathcal{N}(x; 6\nu, 4\omega)}$$

For $z = 1$:

$$p(z = 1 \mid x, \theta) = \frac{(1 - \alpha)\mathcal{N}(x; 6\nu, 4\omega)}{\alpha\mathcal{N}(x; 2\nu, 5\omega) + (1 - \alpha)\mathcal{N}(x; 6\nu, 4\omega)}$$

c. **M-Step:** To make an estimate on our parameters, we can use the maximum likelihood estimator. Making the standard assumption that data are i.i.d., likelihood can be expressed as:

$$\mathcal{L}(\theta; d) = p(d \mid \theta) = \prod_{n=1}^{N} p(d^{\{n\}} \mid \theta)$$

Unfortunately, this won't lend itself easily to optimization. Instead we'll maximize an evidence lower bound (ELBO) of the log-likelihood that is tight around the current values of the parameter set[1].

$$\ell(\theta; d) \geq \sum_{n=1}^{N} \sum_{k}^{\{0,1\}} \left[p(z = k \mid d^{\{n\}}, \theta_{\text{curr}}) \cdot \ln \left(p(d^{\{n\}}, z = k \mid \theta) \right) \right]$$

Hint

θ_{curr}, here, refers to the values of θ in the current iterative step. You may recognize the term $p(z = k \mid d^{\{n\}}, \theta_{\text{curr}})$ from lecture as responsibility—the probability that a point belongs in a particular cluster. This is what you computed in the E-Step, just applied to our dataset. Notationally, we can let $\tau_{nk} = p(z = k \mid d^{\{n\}}, \theta_{\text{curr}})$. For this step, all τ_{nk} are constants, whose derivative with respect to anything is 0.

From this expression, use the derivative to find the value of ω that maximizes the ELBO. Simplify your result as much as possible, and leave your answer in terms of τ, d, υ, and N. Show your work. [9pts]

The ELBO is:

$$Q(\theta) = \sum_{n=1}^{N} \sum_{k=0}^{1} \tau_{nk} \ln \left(p(d^{\{n\}}, z = k \mid \theta) \right)$$

For $z = 0$:

$$\ln \left(p(d^{\{n\}}, z = 0 \mid \theta) \right) = \ln(\alpha) - \frac{1}{2} \ln(10\pi) - \frac{1}{2} \ln(\omega) - \frac{(d^{\{n\}} - 2\nu)^2}{10\omega}$$

For $z = 1$:

$$\ln \left(p(d^{\{n\}}, z = 1 \mid \theta) \right) = \ln(1 - \alpha) - \frac{1}{2} \ln(8\pi) - \frac{1}{2} \ln(\omega) - \frac{(d^{\{n\}} - 6\nu)^2}{8\omega}$$

1.Substitute these into $Q(\theta)$ 2. group the ω terms 3. drop all constants with respect to ω:

$$Q(\omega) = \sum_{n=1}^{N} \tau_{n0} \left[-\frac{1}{2} \ln(\omega) - \frac{(d^{\{n\}} - 2\nu)^2}{10\omega} \right] + \tau_{n1} \left[-\frac{1}{2} \ln(\omega) - \frac{(d^{\{n\}} - 6\nu)^2}{8\omega} \right] + C$$

4. Take the derivative with respect to ω 5. set it to 0 to find the maximum:

$$\frac{\partial Q}{\partial \omega} = \sum_{n=1}^{N} \tau_{n0} \left[-\frac{1}{2\omega} + \frac{(d^{\{n\}} - 2\nu)^2}{10\omega^2} \right] + \tau_{n1} \left[-\frac{1}{2\omega} + \frac{(d^{\{n\}} - 6\nu)^2}{8\omega^2} \right] = 0$$

6. Multiply the entire equation by $2\omega^2$:

$$\sum_{n=1}^{N} \tau_{n0} \left[-\omega + \frac{(d^{\{n\}} - 2\nu)^2}{5} \right] + \tau_{n1} \left[-\omega + \frac{(d^{\{n\}} - 6\nu)^2}{4} \right] = 0$$

$$\sum_{n=1}^{N} (-\omega)(\tau_{n0} + \tau_{n1}) + \sum_{n=1}^{N} \left(\frac{\tau_{n0}(d^{\{n\}} - 2\nu)^2}{5} + \frac{\tau_{n1}(d^{\{n\}} - 6\nu)^2}{4} \right) = 0$$

Since sum(responsibilities) === 1, the first summation is $-\omega N$:

$$\omega N = \sum_{n=1}^{N} \left(\frac{\tau_{n0}(d^{\{n\}} - 2\nu)^2}{5} + \frac{\tau_{n1}(d^{\{n\}} - 6\nu)^2}{4} \right)$$

$$\omega = \frac{1}{N} \sum_{n=1}^{N} \left(\frac{\tau_{n0}(d^{\{n\}} - 2\nu)^2}{5} + \frac{\tau_{n1}(d^{\{n\}} - 6\nu)^2}{4} \right)$$

[1] A derivation of this ELBO can be found in the appendices of this document.

2.2 Gradient Ascent Comparison [1% Bonus For All]

EM cannot be applied to every problem, in that it only applies to a specific problem structure: one with latent variables. In the GMM problem, the hidden variable is the responsibilities. If we knew the ground truth responsibilities, we could simply use MLE to find the optimal model instantly. EM leverages this structure to get an efficient solution. However, there are other tools we could have used.

Gradient ascent/descent is a far more general optimization technique, in that it can work on any problem where you have a differentiable objective function (which is most problems in ML). To apply it to this problem, we could analytically compute the gradient of the log-likelihood with respect to the parameters, then take a small step in the parameter space in a direction that would increase the log-likelihood. Eventually, this process will find a maximum.

GA might be favorable here, not to mention that there's a veritable ocean of literature on improving gradient ascent/descent. It's not really leveraging specifics of the problem, but it's a prolific algorithm. We could just as easily solve this problem with gradient ascent, and maybe it would do it better.

$$\theta = \theta_{\text{curr}} + \eta \nabla \mathcal{L}(\theta_{\text{curr}})$$

What is one specific advantage that EM might have over gradient ascent for the problem explored in the previous section, that is to say, what is one reason you might choose EM over GA? Justify your answer with evidence. Either show your workings or cite from peer-reviewed literature. [1% Bonus For All]

One specific advantage of EM over GA is that EM requires valid parameter constraints without requiring learning rate tuning or bounds projection. For exmaple, in our GMM, ω acts as a variance scaling parameter, meaning we have a strict mathematical constraint that $\omega > 0$. In GA, an update step $\theta = \theta_{\text{curr}} + \eta \nabla \mathcal{L}(\theta_{\text{curr}})$ result in a negative value for ω if the gradient is steep or the step size is too large. A negative ω renders the Gaussian distribution undefined and breaks the model. In contrast, the closed-form M-step update we derived for ω is a summation of squared terms multiplied by positive probabilities (τ_{nk}). This guarantees that ω remains strictly positive at every iterative step.

3 Numeric Stability [6pts]

In the Programming assignment (section 2.3), you should have implemented the helper functions for GMM. The notebook contains some information which may be useful in answering these questions.

a. In softmax and logsumexp, we subtracted the row maximums from unnormalized model outputs in the E-step of GMM. Explain how overflow/underflow can arise in these situations and effect the EM algorithm's convergence. Justify your answer (numerically, or brief explanation). [2pts]

Overflow: If the logit values are large and positive, exp(logit) will easily exceed the maximum representable floating point number, resulting in infinity. This cascades into NaNs when doing division in softmax, causing the EM algorithm to immediately crash or diverge.
Underflow: If the logit values are very large and negative, exp(logit) will round to 0. In softmax, this can lead to a zero division if all terms in a row underflow. In logsumexp, taking log(0) results in -inf.

By subtracting the row maximum $m = \max(l)$ from the logits, the largest value passed to exp becomes exactly 0. This prevents overflow since all inputs to exp ≤ 0. It also prevents disastrous underflow because the sum in the denominator of softmax or inside the log of logsumexp is guaranteed to be at least 1, safely avoiding division by zero and log(0).

b. In the implementation instructions for logsumexp, you are told to subtract the row maximums, calculate logsumexp, then add them back. Prove (algebraically) how this process will return a correct result. More formally, let l be a row of D logits:

$$l = \{l_1, l_2, \cdots, l_D\}$$

Let s^* be the result of logsumexp applied directly to l:

$$s^* = \log\left(\sum_{i=1}^{D} \exp(l_i)\right)$$

Let s' be the result of logsumexp with our technique, subtracting the max before, adding the max after.

$$s' = \log\left(\sum_{i=1}^{D} \exp\left(l_i - \max_{j\in[1,D]}[l_j]\right)\right) + \max_{j\in[1,D]}[l_j]$$

You need to show that $s' = s^*$. [4pts]
Let $m = \max_{j\in[1,D]}[l_j]$

$$s' = \log\left(\sum_{i=1}^{D} \exp(l_i - m)\right) + m$$

$$s' = \log\left(\sum_{i=1}^{D} \exp(l_i)\exp(-m)\right) + m$$

$$s' = \log\left(\exp(-m)\sum_{i=1}^{D} \exp(l_i)\right) + m$$

$$s' = \log(\exp(-m)) + \log\left(\sum_{i=1}^{D} \exp(l_i)\right) + m$$

Since $\log(\exp(-m)) = -m$:

$$s' = -m + s^* + m$$

$$s' = s^*$$

This proves that subtracting the maximum before the operation and adding it back after yields the exact same mathematical result

4 Alternate Implementation [5pts]

In the Programming assignment (section 2.5.1), you should have implemented a compression scheme for images using GMM clustering. The notebook contains some information which may be useful in answering this question.

Our compression scheme was dependent on clustering, but GMM isn't the only clustering solution. E.g., in section 1, you implemented K-means clustering. Consider how the clustering process would have been different if you had used K-means instead. Is there any way in which K-means might be more or less effective than GMM for this specific application? [5pts]

If we had used K-Means instead of GMM for compressing these images, the main thing that changes is how we assign colors. Specifically, hard versus soft clustering. Honestly, K-Means is a lot more practical for this specific job because it is way faster. When you flatten an image, you are dealing with hundreds of thousands or even millions of pixels as your data points. GMM has to do a ton of heavy math for every single one of them calculating covariance matrix inverses, determinants, and complex exponentials. K-Means just checks the basic Euclidean distance, which saves a massive amount of computation time.

Where GMM actually is useful is in capturing the real shape of color distributions. K-Means basically assumes every color cluster is perfectly round. But think about a photo of a sunset: the colors blend in long, stretched-out gradients. GMM's covariance matrices allow it to stretch and morph to fit those elliptical color groupings much better.

However, for our specific compression scheme, we end up throwing away GMM's best feature anyway. At the very end, we force every single pixel to turn into the exact mean color (μ) of its top cluster. Since we are making a strict "hard" assignment at the end regardless, doing all that complex, probabilistic "soft" clustering with GMM is kind of overkill. K-Means gets us straight to that hard assignment much more naturally and with way less math.

5 Semi-supervised Learning [2.0% Bonus For All]

This section is accompanied by the semi-supervised data imputation part of the programming assignment (section 3.1).

1. What are some noticeable differences in the values produced by kNN imputation method, versus median value imputation? [0.5%]

   The primary difference lies in how they utilize the dataset's structure. KNN imputation is a multivariate, local approach that estimates missing values based on the similarity of surrounding data points. Consequently, it preserves relationships between different features and maintains the local variance of the data. Conversely, median imputation is a univariate, global approach that replaces every missing value in a feature with a single constant. This ignores any correlation with other features, artificially reduces the overall variance of that feature, and can severely distort the underlying multivariate distribution.

2. We could have easily used the mean of each column to impute instead of the median. What advantage might median offer over mean for this use case? [0.5%]

   The principal advantage of the median over the mean is its robustness to outliers and highly skewed distributions. The mean is highly sensitive to extreme values; a few massive outliers can significantly shift the average, resulting in an imputed value that is not representative of the typical data point. The median, representing the 50th percentile, remains stable even in the presence of severe outliers, making it a safer and more reliable metric for central tendency imputation in raw, unvetted datasets.

3. Complete Case Analysis (CCA) is an alternative to imputation: simply drop every row that has a NaN. You only analyze the complete datapoints. This technique is not favored among statisticians. CCA is only justifiable on data that is Missing Completely At Random, i.e., the missingness itself is scatterred independently of the data. Why might Complete Case Analysis be unfavorable when the missingness is dependent on the values of the data itself? [1.0%]

   When data missingness depends on the data values themselves, CCA introduces systemic bias into the dataset. By discarding incomplete rows, CCA removes a non-random, specific subset of the population. Consequently, the surviving complete dataset is no longer a representative sample of the true underlying distribution. Any statistical models or insights derived from this skewed subset will be fundamentally biased and fail to generalize accurately to the broader population.

6 K-Selection Problem [3% Bonus For All]

In the Programming assignment (section 1.1-1.3), you should have implemented the backbone of k-Means. The notebook contains some information which may be useful in answering this question.

In practice, the hardest part about using k-Means is picking an optimal value for k. Your assignment includes a data file "clusterable.csv", on which you'll attempt to do k-Means clustering. Read through the section informing you about how to evaluate clustering metrics in the notebook (section 1.4).

Your task is to estimate the optimal value of k for "clusterable.csv". To do this, you should pick any clustering metric, apply that clustering metric across a range of possible values for k, plot those results in a line plot (metric vs. k-value), embed the line plot into this document, and then provide an analysis of the graph and what k-value it suggests. Note that using only one metric may not be sufficient to support your answer. If you do this for real data in the real world, you'll want to use every tool available to you, and the tools will often disagree.

To receive credit, you should:

- For each metric:
  - Name the metric, briefly explain the metric, and mention what values would suggest a good k on that metric.
  - Embed a plot of the metric's performance vs. proposed k-values.
  - Analyze the plot. Make a guess for k based on the plot and briefly explain how confident this guess is.
  - Remember, negative results are good too! Even if your plot tells you nothing, it's still useful to a reader to see what you tried. You can simply ignore a useless result, but you should still include it.
- If you find your first metric to not be so convincing, you should repeat those steps for new metrics. Once you're satisfied with your answer, analyze all of the individual results and make a final recommendation based on some combination of the results.
- With your chosen value of k, cluster the data with that k-value one more time, make a scatter-plot with each cluster colored differently and embed that here. [3% Bonus For All]

To estimate the optimal number of clusters, I evaluated the kmeans algorithm for k values ranging from 2 to 14. Because K-Means is sensitive to initialization and can settle into poor local minima, I recorded the best performing model across multiple runs for each value. I then analyzed the results using two different clustering metrics: WCSS and the Silhouette Coefficient.

1.
WCSS calculates the total squared distance from each data point to its assigned cluster centroid. Because this metric inherently decreases as k increases, we cannot simply look for the lowest value. Instead, I used the elbow method to find the point of diminishing returns.

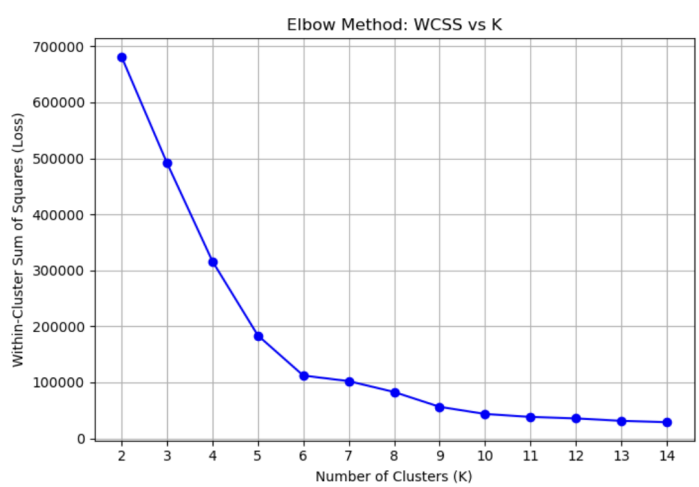

Figure 1: WCSS vs Number of Clusters

Looking at the WCSS plot, there is a steep, rapid decline in error from $k = 2$ to $k = 6$. At exactly $k = 6$, the graph exhibits a sharp elbow and flattens out into a gradual slope. This indicates that adding a 7th cluster or beyond provides very little benefit, suggesting 6 is a strong candidate.

2.
To verify this, I looked at the Silhouette Coefficient. This metric evaluates how closely a point fits within its own cluster compared to neighboring clusters. A score closer to 1 suggests tight, well-separated clusters.

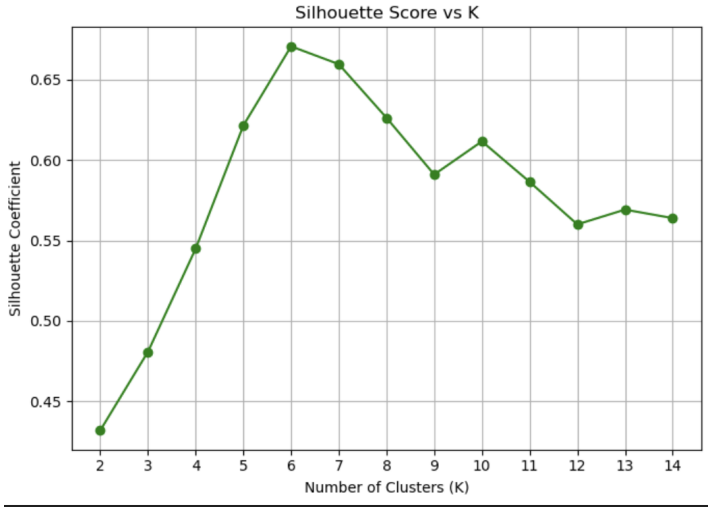

Figure 2: Silhouette Coefficient vs Number of Clusters

The Silhouette plot strongly supports the conclusion drawn from the elbow method. The score climbs steadily and hits a distinct global maximum at 6 (with a score of 0.67). After this peak, the score drops off, indicating that higher values of k are artificially splitting natural groupings.

3. (ANSWER)
Therefore, 6 is likely to be the optimal number of clusters. While different clustering metrics can often disagree on real-world data, the sharp elbow in the WCSS plot perfectly aligns with the absolute peak of the Silhouette score, providing strong evidence for this configuration.

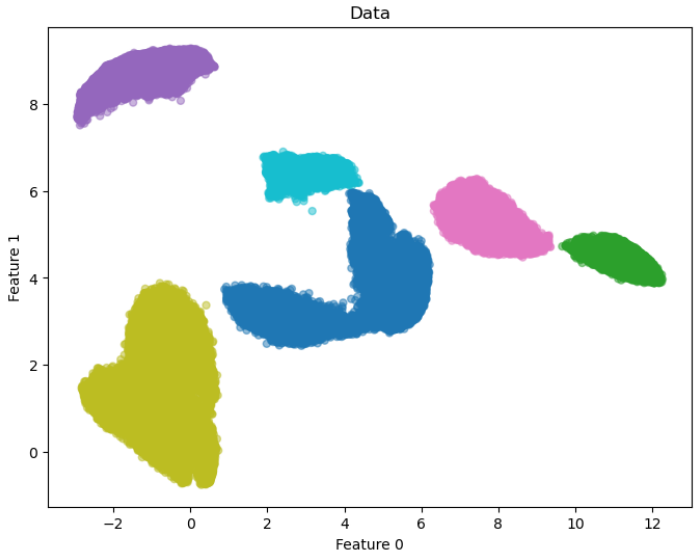

Figure 3: Final K-Means Clustering Assignment ($k = 6$)

Appendix

The appendix will include any information that is extraneous to the questions asked, but still informative. This content is not necessary for the homework, only if you're curious about further math.

Justification for the ELBO

In the implementation of GMM, we maximize a tight lower bound, so-called, Evidence Lower BOund (ELBO). This is because the traditional MLE approach isn't very approachable—maybe numerically, but not analytically.

$$\begin{aligned}
\mathcal{L}(\theta; d) &= \prod_{n=1}^{N} p(d^{\{n\}} \mid \theta) \\
&= \prod_{n=1}^{N} \sum_{k=1}^{K} p(d^{\{n\}}, z_k \mid \theta) \\
\ell(\theta; d) &= \ln \left[\prod_{n=1}^{N} \sum_{k=1}^{K} p(d^{\{n\}}, z_k \mid \theta) \right] \\
&= \sum_{n=1}^{N} \ln \left[\sum_{k=1}^{K} p(d^{\{n\}}, z_k \mid \theta) \right]
\end{aligned}$$

Log of a sum won't give a clean gradient to zero out, but Jensen's might help us commute the sum through the log. That probability can be expanded (since $d^{\{n\}}$ and z_k are conditionally independent), but even if we got something more Jensen's like, the weights would still be conditioned on θ. We can't use variable weights, even if they are guaranteed to sum to one, they need to be constants for Jensen's (θ is a variable of the function). We can introduce a term dependent on the current parameters θ_{curr} (that sums to 1 over K), and maybe make some guarantees about the update range of the parameters.

$$\begin{aligned}
\ell(\theta; d) &= \sum_{n}^{N} \ln \left[\sum_{k}^{K} \left(\frac{p(z_k \mid d^{\{n\}}, \theta_{curr})}{p(z_k \mid d^{\{n\}}, \theta_{curr})} \right) \cdot p(d^{\{n\}}, z_k \mid \theta) \right] \\
&= \sum_{n}^{N} \ln \left(\sum_{k}^{K} p(z_k \mid d^{\{n\}}, \theta_{curr}) \cdot \frac{p(d^{\{n\}}, z_k \mid \theta)}{p(z_k \mid d^{\{n\}}, \theta_{curr})} \right) \\
&\quad \text{via total probability, } \textstyle\sum_k^K p(z_k \mid \cdots) = 1 \\
&\quad \text{via second derivative test, ln is concave} \\
&\quad \text{thus, } \ln(\mathbb{E}[x]) \geq \mathbb{E}[\ln(x)] \\
\therefore \ell(\theta; d) &\geq \sum_{n}^{N} \sum_{k}^{K} p(z_k \mid d^{\{n\}}, \theta_{curr}) \cdot \ln \left(\frac{p(d^{\{n\}}, z_k \mid \theta)}{p(z_k \mid d^{\{n\}}, \theta_{curr})} \right) \\
&= \sum_{n}^{N} \sum_{k}^{K} p(z_k \mid d^{\{n\}}, \theta_{curr}) \cdot \left(\ln p(d^{\{n\}}, z_k \mid \theta) - \ln p(z_k \mid d^{\{n\}}, \theta_{curr}) \right)
\end{aligned}$$

We can further relax this lower bound by simply dropping the second log term. Since $p(z_k|...)$ is a probability and we've already implicitly assumed that it's never 0 or 1 in our E-step, which would mean that one of the components is useless (which is not possible for a Gaussian anyways), then we know $p(z_k \mid d^{\{n\}}, \theta_{curr}) \in (0, 1)$. Thus, $\ln\ p(z_k \mid d^{\{n\}}, \theta_{curr}) \leq 0$. Since this term is always negative, and it's currently being subtracted, dropping this term would strictly decrease the value, which is just a loosening of the lower bound.

$$\begin{aligned}
\ell(\theta; d) &\geq \sum_{n}^{N} \sum_{k}^{K} p(z_k \mid d^{\{n\}}, \theta_{curr}) \cdot \ln \left(p(d^{\{n\}}, z_k \mid \theta) \right) \\
&\quad \text{definition of E-step's tau, verbatim} \\
\ell(\theta; d) &\geq \sum_{n}^{N} \sum_{k}^{K} \tau_{nk} \cdot \ln \left(p(d^{\{n\}}, z_k \mid \theta) \right) \\
&\quad \text{for notation's sake, call it} \\
\ell(\theta; d) &\geq Q(\theta, \theta_{\text{curr}})
\end{aligned}$$

And so, we get this function, Q, which is a valid lower bound on the log-likelihood ($\ell(\theta; d) \geq Q(\theta, \theta_{\text{curr}})$), it's evidenced by the E-step's estimate, and also, it should be tight if you don't go anywhere ($\ell(\theta_{\text{curr}}; d) = Q(\theta_{\text{curr}}, \theta_{\text{curr}})$).

This is all well and good, but how can we be sure this iterative process will actually work? Honestly the math is above my head quite a bit, but I was able to prove that the optimal value for θ is shared, so if the current parameters constitute a locally maximum likelihood for the GMM, the EM process will stay stationary.

We can prove that with contradiction. To begin, assume the opposite of the implication ($p \wedge \neg q$):

- "the current parameters constitute a locally maximum likelihood for the GMM" means that θ_{curr} is a local maximum for $\ell(\theta; d)$
- "the EM process will **NOT** stay stationary" means that θ_{curr} is not the global maximum for $Q(\theta, \theta_{\text{curr}})$ and the impending M-step would update $\theta = \arg_\theta \max Q(\theta, \theta_{\text{curr}})$. This implies that there exists some point θ' in an arbitrarily small ϵ-neighborhood around θ_{curr} which satisfies $\mathcal{Q}(\theta', \theta_{\text{curr}}) \geq \mathcal{Q}(\theta_{\text{curr}}, \theta_{\text{curr}})$.

The existence of θ' is not general, but we can choose it to be arbitrarily close to θ_{curr}. Since θ_{curr} is a local maximum for $\ell(\theta; d)$ and we can get arbitrarily close, we can presume $\ell(\theta_{\text{curr}}; d) > \ell(\theta'; d)$.

Next, recall that the ELBO is tight at the current parameters, so $\ell(\theta_{\text{curr}}; d) = Q(\theta_{\text{curr}}, \theta_{\text{curr}})$.

Finally, all of these observations link into a syllogism:

$$
\begin{aligned}
&\mathcal{Q}(\theta', \theta_{\text{curr}}) \geq \mathcal{Q}(\theta_{\text{curr}}, \theta_{\text{curr}}) = \ell(\theta_{\text{curr}}; d) > l(\theta'; d) \\
&\mathcal{Q}(\theta', \theta_{\text{curr}}) > \ell(\theta'; d) \\
&\qquad\qquad \text{however, we know } Q \text{ to be an ELBO for any input} \\
&\mathcal{Q}(\theta', \theta_{\text{curr}}) \overset{!}{\leq} \ell(\theta'; d)
\end{aligned}
$$

Thus, a contraction is shown, so the assumption was false, and the implication "if the current parameters constitute a locally maximum likelihood for the GMM, the EM process will stay stationary" is proven true. $\square$

If the EM algorithm ascends to a local max with the surrogate bound, it will stay there, but we haven't covered the if. Convergence is guaranteed, since likelihood is guaranteed to monotonically increase at each iteration.